Homework 1

Davide Quartucci
Student ID: 2191105

Exercise 1. Use the 4-stage Runge-Kutta method to solve the equation

$$\begin{cases} y'(t) = -10y^2(t) \\ y(0) = 1 \end{cases} \tag{1}$$

$t \in (0, 2)$ with $h = 2^{-k}$, $k = 5, \ldots, 10$. For all (six) values of h solve the equation and compute the error at the final time $T = 2$. Provide a loglog plot of this error as a function of the number of steps. Produce also a table with the values of h and of the error in each row. Check if the error reduces (progressively halving h) in accordance with the theory.

Solution. We solve the initial value problem using the explicit Runge-Kutta method of order 4 (RK4). The simulation was performed over the interval $t \in [0, 2]$ with decreasing step sizes $h = 2^{-k}$ for $k = 5, \ldots, 10$.

We formulate the algorithm using the generic vector field notation $f(t, y)$. Let h be the time step and $t_n = t_0 + nh$ be the discrete time nodes. The update scheme for the step $t_n \to t_{n+1}$ is defined as:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{2}$$

where the four stages k_i are computed as:

$$k_1 = f(t_n, y_n) \tag{3}$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \tag{4}$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \tag{5}$$

$$k_4 = f(t_n + h, y_n + hk_3) \tag{6}$$

In our specific case, we have to consider $f(t, y) = -10y^2$.

To evaluate the accuracy at the final time $T = 2$, we compare the numerical result with the exact analytical solution of the ODE, which is:

$$y_{true}(t) = \frac{1}{10t + 1} \tag{7}$$

The absolute errors computed as $|y_n(T) - y_{true}(T)|$ are reported in Table 1, while Figure 1 shows the log-log plot of the error as a function of the number of steps.

The RK4 method is of order 4. Theoretically, this implies that the global truncation error is proportional to h^4 ($Error \approx C \cdot h^4$). When the step size h is halved ($h \to h/2$), the expected error becomes:

$$Error_{\text{new}} \approx C \cdot \left(\frac{h}{2}\right)^4 = C \cdot \frac{h^4}{16} = \frac{Error_{\text{old}}}{16}.$$

Therefore, the ratio $\frac{Error_{\text{old}}}{Error_{\text{new}}}$ should approach 16 as h decreases. The experimental results confirm this expectation, as shown in Table 2, demonstrating the correct convergence properties of the method.

Table 1: Error analysis at $T = 2$

k	h	Error
5	0.031250	2.291845e-07
6	0.015625	1.785764e-08
7	0.007812	1.160234e-09
8	0.003906	7.312863e-11
9	0.001953	4.579587e-12
10	0.000977	2.863751e-13

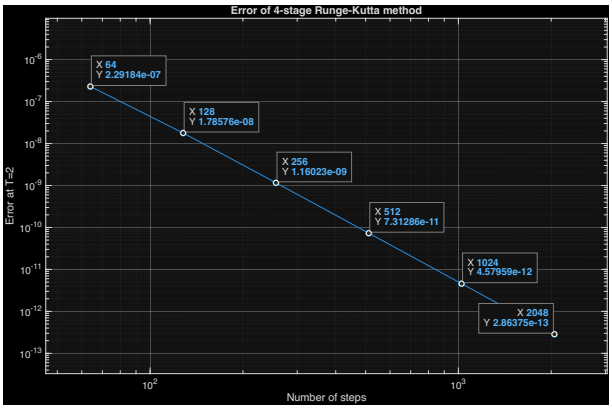


Figure 1: Log-log plot of the error vs number of steps.

Table 2: Error reduction check. The theoretical ratio is $2^4 = 16$.

Ratio of errors for consecutive h	Ratio
0.031250 / 0.015625	12.83
0.015625 / 0.007812	15.39
0.007812 / 0.003906	15.87
0.003906 / 0.001953	15.97
0.001953 / 0.000977	15.99

The script I used for the simulations and data generation is provided in the attached file `matlab_script/runge_kutta_solver.m`. □

Exercise 2. A BDF formula is obtained by first writing the ODE on a point t_{n+k} as $y'(t_{n+k}) = f(t_{n+k}, y_{n+k})$. Then the unknown function y is interpolated at the left hand side of $y' = f(t, y)$ using the nodes $t_n, \dots, t_{n+k}$ and then differentiated. Obtain the BDF2 formula ($k = 2$) and find its local truncation error (Hint: recall that the interpolation error is

$$E(t) = \frac{(t - t_{n+k}) \dots (t - t_n) f^{(k+1)}(\eta(t))}{(k+1)!}.$$

To obtain the local truncation error you should find the maximum of the derivative of $E'(t)|$ which usually is taken at the endpoints of the interval). Prove that BDF2 is absolutely stable in the real interval $\overline{h} \in (-\infty, 0)$. (Hint: the characteristic polynomial for this method applied to the test equation is

$$t^2(3 - 2\overline{h}) - 4t + 1 = 0$$

with roots

$$t_{1,2} = \frac{2 \pm \sqrt{1 + 2\overline{h}}}{3 - 2\overline{h}}$$

if $-\frac{1}{2} \leq \overline{h} < 0$ then $t_{1,2}$ are both real and... If, otherwise, $\overline{h} < \frac{1}{2}$ then $t_{1,2}$ is complex and its modulus is...)

Solution. 1. **Derivation of the BDF2 Formula**

To derive the BDF2 method, we approximate the solution $y(t)$ with a quadratic polynomial $P(t)$ interpolating the values (t_n, y_n), (t_{n+1}, y_{n+1}), (t_{n+2}, y_{n+2}) where t_{n+2} is the point of interest, $t_{n+1} = t_{n+2} - h$ and $t_n = t_{n+2} - 2h$.

We construct the unique interpolating polynomial $P(t)$ of degree 2 using Lagrange:

$$P(t) = y_n L_n(t) + y_{n+1} L_{n+1}(t) + y_{n+2} L_{n+2}(t)$$

where $L_j(t)$ are the Lagrange basis polynomials. Explicitly writing the basis polynomials for our defined nodes:

$$L_{n+2}(t) = \frac{(t-t_{n+1})(t-t_n)}{(t_{n+2}-t_{n+1})(t_{n+2}-t_n)} = ... = \frac{(t-t_{n+1})(t-t_n)}{2h^2}$$
$$L_{n+1}(t) = \frac{(t-t_{n+2})(t-t_n)}{(t_{n+1}-t_{n+2})(t_{n+1}-t_n)} = ... = -\frac{(t-t_{n+2})(t-t_n)}{h^2}$$
$$L_n(t) = \frac{(t-t_{n+2})(t-t_{n+1})}{(t_n-t_{n+2})(t_n-t_{n+1})} = ... = \frac{(t-t_{n+2})(t-t_{n+1})}{2h^2}$$

The approximation of the derivative is given by $y'(t_{n+2}) \approx P'(t_{n+2})$. We compute the derivatives of the basis polynomials and evaluate them at $t = t_{n+2}$:

(a) **Derivative of L_{n+2}:**

$$L'_{n+2}(t) = \frac{(t-t_n)+(t-t_{n+1})}{2h^2} \implies L'_{n+2}(t_{n+2}) = \frac{3}{2h}$$

(b) **Derivative of L_{n+1}:**

$$L'_{n+1}(t) = -\frac{(t-t_n)+(t-t_{n+2})}{h^2} \implies L'_{n+1}(t_{n+2}) = -\frac{4}{2h}$$

(c) **Derivative of L_n:**

$$L'_n(t) = \frac{(t-t_{n+1})+(t-t_{n+2})}{2h^2} \implies L'_n(t_{n+2}) = \frac{1}{2h}$$

Finally, substituting these values into the derivative:

$$P'(t_{n+2}) = y_n\left(\frac{1}{2h}\right) + y_{n+1}\left(-\frac{4}{2h}\right) + y_{n+2}\left(\frac{3}{2h}\right) = \frac{3y_{n+2} - 4y_{n+1} + y_n}{2h}$$

Setting $P'(t_{n+2}) = f(t_{n+2}, y_{n+2})$, we obtain rearranging the standard BDF2 formula:

$$3y_{n+2} - 4y_{n+1} + y_n = 2hf_{n+2}$$

2. **Local Truncation Error (LTE)**

According to the theory, the BDF2 method approximates the derivative $y'(t_{n+2})$ with the derivative of the interpolating polynomial $P'(t_{n+2})$. Thus, the local truncation error T_{n+2} is defined as the difference between the numerical approximation and the exact derivative:

$$T_{n+2} = P'(t_{n+2}) - y'(t_{n+2})$$

Let $E(t) = y(t) - P(t)$ be the interpolation error. Then $T_{n+2} = -E'(t_{n+2})$. From interpolation theory, the error for a polynomial of degree $k = 2$ interpolating at nodes $\{t_n, t_{n+1}, t_{n+2}\}$ is given by:

$$E(t) = \frac{(t-t_{n+2})\dots(t-t_n)f^{(2+1)}(\eta(t))}{(2+1)!} = \frac{y^{(3)}(\eta(t))}{3!}\omega(t), \quad \omega(t) = (t-t_{n+2})(t-t_{n+1})(t-t_n)$$

To find the error on the derivative $E'(t)$, we differentiate the expression using the product rule:

$$E'(t) = \frac{y^{(3)}(\eta(t))}{6}\omega'(t) + \omega(t)\frac{d}{dt}\left(\frac{y^{(3)}(\eta(t))}{6}\right)$$

We evaluate this at the point $t = t_{n+2}$. Notice that $\omega(t_{n+2}) = 0$ because the term $(t - t_{n+2})$ vanishes. Therefore, the second part of the derivative disappears, and we are left with:

$$E'(t_{n+2}) = \frac{y^{(3)}(\eta)}{6}\omega'(t_{n+2})$$

Now we calculate $\omega'(t)$. Letting $\omega(t) = (t - t_{n+2})(t - t_{n+1})(t - t_n)$, the derivative is:

$$\omega'(t) = (t - t_{n+1})(t - t_n) + (t - t_{n+2})(t - t_n) + (t - t_{n+2})(t - t_{n+1})$$

Evaluating at $t = t_{n+2}$ (where terms with t_{n+2} vanish):

$$\begin{aligned}\omega'(t_{n+2}) &= (t_{n+2} - t_{n+1})(t_{n+2} - t_n) + 0 \cdot 0 + 0 \cdot 0 \\ &= (h)(2h) \\ &= 2h^2\end{aligned}$$

Substituting this back into the expression for $E'(t_{n+2})$:

$$E'(t_{n+2}) = \frac{y^{(3)}(\eta)}{6}(2h^2) = \frac{1}{3}h^2 y^{(3)}(\eta)$$

Finally, recalling that $T_{n+2} = -E'(t_{n+2})$, the local truncation error is:

$$T_{n+2} = -\frac{1}{3}h^2 y^{(3)}(\eta)$$

This confirms that the BDF2 method is of order 2 ($O(h^2)$) and matches the result obtained.

3. **Absolute Stability**

To analyze absolute stability, we apply the BDF2 method to the test equation $y' = \lambda y$:

$$3y_{n+2} - 4y_{n+1} + y_n = 2h(\lambda y_{n+2})$$

Let $\bar{h} = h\lambda$. Rearranging the terms gives the linear recurrence relation:

$$(3 - 2\bar{h})y_{n+2} - 4y_{n+1} + y_n = 0$$

We look for solutions of the form $y_n = t^n$. This leads to the characteristic polynomial. In addiction we denote the roots of the polynomial as t (not to be confused with the time variable):

$$(3 - 2\bar{h})t^2 - 4t + 1 = 0$$

The method is absolutely stable if the roots $t_{1,2}$ satisfy $|t| \leq 1$. Applying the quadratic formula:

$$t_{1,2} = \frac{4 \pm \sqrt{16 - 4(3 - 2\bar{h})}}{2(3 - 2\bar{h})} = \frac{2 \pm \sqrt{1 + 2\bar{h}}}{3 - 2\bar{h}}$$

We analyze the roots for $\bar{h} \in (-\infty, 0)$. Since $\bar{h}$ is negative, the denominator $3 - 2\bar{h}$ is always positive and greater than 3.

Case 1: Real Roots ($-\frac{1}{2} \leq \bar{h} < 0$) Here the discriminant $1 + 2\bar{h} \geq 0$, so the roots are real.

- At $\bar{h} = 0$, the roots are $t_1 = 1$ and $t_2 = 1/3$.
- At the transition point $\bar{h} = -1/2$, the discriminant is zero, yielding a double root $t_{1,2} = \frac{2}{4} = 1/2$.

Since the roots are positive and ≤ 1 in this interval, the numerical solution exhibits a **monotonic decay** towards zero, closely mimicking the behavior of the exact solution $e^{\lambda t}$.

Case 2: Complex Roots ($\bar{h} < -\frac{1}{2}$) Here $1+2\bar{h} < 0$, so the roots are complex conjugates:

$$t_{1,2} = \frac{2 \pm i\sqrt{-(1+2\bar{h})}}{3-2\bar{h}}$$

We compute the squared modulus $|t|^2 = t \cdot \bar{t}$:

$$|t|^2 = \frac{4 + (-(1+2\bar{h}))}{(3-2\bar{h})^2} = \frac{3-2\bar{h}}{(3-2\bar{h})^2} = \frac{1}{3-2\bar{h}}$$

In this interval, $-2\bar{h} > 1$, so the denominator $3 - 2\bar{h} > 4$. Consequently:

$$|t|^2 < \frac{1}{4} \implies |t| < \frac{1}{2}$$

This confirms absolute stability. Since $|t| \to 0$ as $\bar{h} \to -\infty$, the method is strict A-stable. This ensures strong damping of high-frequency components relative to large negative eigenvalues.

□

Exercise 3. We want to solve the linear system of ODES

$$y'(t) = -Ay(t), \quad y(0) = [1, 1, \ldots, 1]^T$$

with A (5-points Finite Difference discretization of the 2D Laplacian) generated through the following Matlab commands

```
nx=60;
G = numgrid('S',nx);
A = delsq(G)*(nx-1)^2,
```

and $t \in (0, 0.1]$. The exact solution to this problem at the final time is

$$\hat{y} = \exp(-0.1A)y_0,$$

which can be computed via the Matlab function `expm`. Denote by λ the eigenvalue of A with largest modulus (in Matlab this can be computed using the command `lambda=-eigs(A, 1, 'lm')` which delivers minus the largest eigenvalue of A.)

(a) Compute the interval of absolute stability of the 4th order Runge-Kutta method for this problem. Then solve the problem by `ode45` with $t_0 = 0$, $T = 0.1$. Compute the final error (the infinity norm of the difference between the exact and the approximate solution), and the CPU time elapsed (use e.g. `tic` and `toc` commands).

(b) Solve the given problem with the Crank Nicolson (CN) method with $h = 2^{-k}$, with $k = 5, \ldots, 12$ recording for each case the final error obtained and the CPU time.

(c) Solve the given problem with the BDF3 method using $h = 2^{-k}$ with $k = 5, \ldots, 12$, recording for each case the final error obtained and the CPU time. Describe how you choose to compute the first 2 steps y_1 and y_2, needed for the BDF3 method.

Remark. With methods CN and BDF3 a linear system must be solved at each timestep. You must solve it with both the Cholesky factorization (direct method) or by the Conjugate Gradient method with tolerance proportional to the local truncation error. This method will be described in the course on the last decade of October and is available in Matlab as `pcg`.

Provide a table reporting, for each of the 8 runs, the method, the number of steps employed, the error obtained, and the CPU time elapsed. Are the error for CN and BDF3 consistent with the theory? What is the best method for a given accuracy? Provide in the same figure a loglog plot of the error for both methods as a function of the stepsize.

Solution. Let's start by defining the problem parameters and the matrix A in Matlab. The matrix A corresponds to the discrete negative Laplacian operator on a 2D grid, scaled by $(n_x-1)^2$. The exact solution at $T = 0.1$ is computed via the matrix exponential $\hat{y} = \exp(-0.1A)y_0$.

We proceeded with the three methods requested in the exercise as follows:

(a) **4th Order Runge-Kutta Method (RK4)**
We first analyzed the stability constraints. First, we computed the eigenvalue of A with the largest modulus using `eigs`, obtaining:

$$|\lambda_{max}| \approx 2.7828 \times 10^4$$

Since the absolute stability region of the explicit RK4 method extends to approximately -2.78 on the negative real axis, the stability condition $|h\lambda| \leq 2.78$ dictates a severe restriction on the step size:

$$h_{limit} \leq \frac{2.78}{|\lambda_{max}|} \approx 9.9898 \times 10^{-5}$$

We then solved the problem using the adaptive solver `ode45` with strict tolerances. The solver successfully integrated the system, but the stiffness forced it to take small steps to maintain stability. The computed performance metrics are:

```
ode45 final error: 2.1642e-05
ode45 CPU time: 0.2987 seconds
```

Comparing this with the implicit methods (see Table 3), `ode45` is extremely accurate but requires a computational time comparable to the finest grids of the implicit solvers, despite the smoothness of the solution.

(b) **Crank-Nicolson Method (CN)**
Now by using the Crank-Nicolson method, which is A-Stable and of order 2. At each time step, we solve the linear system:

$$y_{n+1} = y_n + \frac{h}{2}\left[(-Ay_n) + (-Ay_{n+1})\right] \implies \left(I + \frac{h}{2}A\right) y_{n+1} = \left(I - \frac{h}{2}A\right) y_n \qquad (8)$$

Since the matrix is Symmetric Positive Definite (SPD), we compared two solvers:

- **Cholesky Factorization:** Computed once outside the loop for efficiency.
- **Conjugate Gradient (PCG):** With tolerance set proportional to the local truncation error (h^3) to avoid over-solving.

(c) **BDF3 Method**

In this part, we implemented the 3-step Backward Differentiation Formula (BDF3), which is Stiffly Stable and of order 3. The update formula requires solving:

$$\frac{11}{6}y_{n+3}-3y_{n+2}+\frac{3}{2}y_{n+1}-\frac{1}{3}y_n = h(-Ay_{n+3}) \implies \left(\frac{11}{6}I + hA\right) y_{n+3} = 3y_{n+2}-\frac{3}{2}y_{n+1}+\frac{1}{3}y_n \tag{9}$$

Startup: Since BDF3 is a multistep method, it requires two previous values y_1, y_2 to start. To maintain A-stability even for the largest step sizes (where explicit methods like RK4 would explode), we computed these starting values using the CN with the same step size h.

Solvers: As with CN, we compared Cholesky factorization and Conjugate Gradient (PCG) with tolerance proportional to the local truncation error.

Results

The following table and plot summarize the error and CPU time for the implicit methods with varying stepsizes $h = 2^{-k}$.

Table 3: Summary of results for CN and BDF3 methods. Note the stability of BDF3 even at large steps thanks to the implicit startup.

	Crank-Nicolson				BDF3			
	Cholesky		CG		Cholesky		CG	
h	Error	Time (s)	Error	Time (s)	Error	Time (s)	Error	Time (s)
0.03125	0.87435	0.0053	0.87435	0.0207	0.20307	0.0041	0.20307	0.0030
0.015625	0.67967	0.0054	0.67967	0.0099	0.02905	0.0044	0.02905	0.0095
0.0078125	0.32303	0.0069	0.32303	0.0189	0.00707	0.0064	0.00707	0.0181
0.0039062	0.04689	0.0113	0.04689	0.0270	0.00704	0.0107	0.00704	0.0255
0.0019531	0.00174	0.0181	0.00174	0.0350	0.00180	0.0189	0.00180	0.0325
0.00097656	0.00178	0.0307	0.00178	0.0395	0.00180	0.0306	0.00180	0.0415
0.00048828	4.50e-4	0.0573	4.50e-4	0.0488	4.47e-4	0.0578	4.47e-4	0.0606
0.00024414	4.48e-4	0.1126	4.48e-4	0.0760	4.47e-4	0.1125	4.47e-4	0.0946

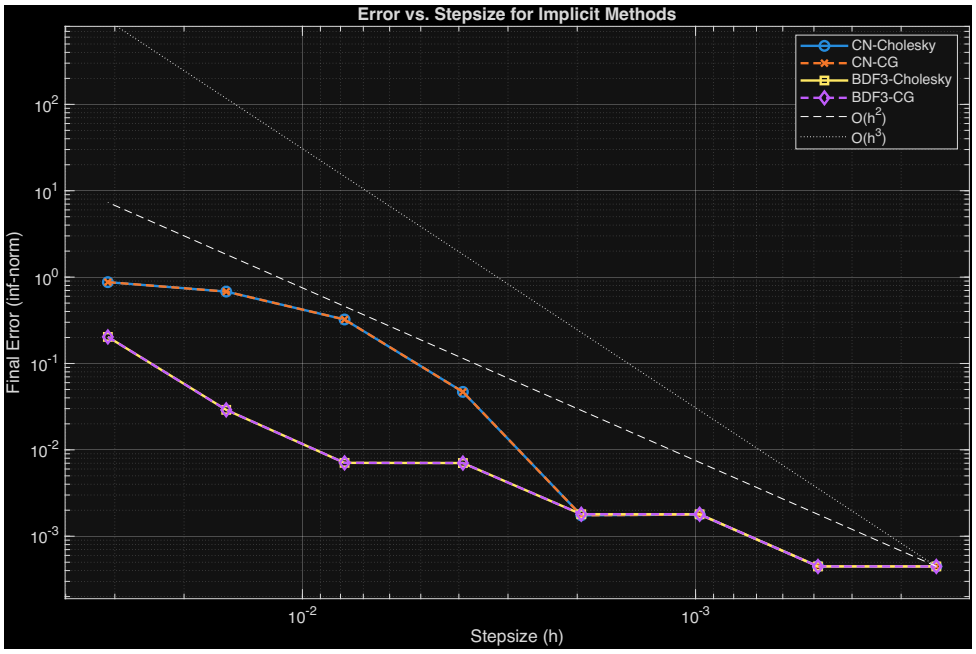


Figure 2: Log-log plot of the error vs stepsize for CN and BDF3 methods with Cholesky and CG solvers.

Consistency with Theory: As shown in Figure 2, the error curves follow the expected slopes.

- **Crank-Nicolson (Blue/Orange):** The method is stable but exhibits a relatively large error for large step sizes (e.g., Error ≈ 0.87 for $h = 0.03$). The error reduction follows the theoretical 2nd order.
- **BDF3 (Yellow/Purple):** With the correct implicit startup, BDF3 is stable throughout the entire range. It significantly outperforms CN: at the coarsest mesh ($h = 0.03$), the error is 0.20 (vs 0.87 for CN), demonstrating the advantage of the 3rd order method.

Best Method and Efficiency:
Comparing with the explicit `ode45` from point (a):

- `ode45` required **0.2987 s** to achieve an error of $\approx 2.1642 \times 10^{-5}$.
- **BDF3 (Cholesky)** achieves an acceptable error (7×10^{-3}) in just **0.0064 s** (at $h = 0.0078$) and reaches the saturation limit (4×10^{-4}) in **0.0578 s**.

The results in Table 3 show that implicit methods are significantly more efficient here. BDF3 achieves a comparable error to ode45 but requires orders of magnitude less CPU time (0.006 s vs 0.29 s)

The script I used for the simulations and data generation is provided in the attached file `matlab_script/ode_system_solver.m`. □

Exercise 4. The Lotka-Volterra equations, also known as the predator-prey equations, are a pair of first-order nonlinear differential equations, frequently used to describe the dynamics of biological systems in which two species interact, one as a predator and the other as prey. The

populations change through time according to the pair of equations:

$$\begin{aligned} x'(t) &= x(t)(\alpha - \beta y(t)) \\ y'(t) &= y(t)(\gamma x(t) - \delta) \\ x(0) &= x_0 \\ y(0) &= y_0 \end{aligned}$$

where

- x is the number of prey (for example, rabbits);
- y is the number of some predator (for example, foxes);
- $x'(t)$ and $y'(t)$ represent the instantaneous growth rates of the two populations;
- t represents time;
- $\alpha, \beta, \gamma, \delta$ are positive real parameters describing the interaction of the two species.

(a) Solve the Lotka-Volterra equation with parameters $\alpha = 0.2$, $\beta = 0.01$, $\gamma = 0.004$, $\delta = 0.07$ by a 4th order RK method using as initial conditions $x_0 = 19$, $y_0 = 22$, $t_0 = 0$, $T = 300$ and a stepsize $h = 10^{-3}$.

(b) Produce a picture with the numerical solution of the prey ($x(t)$) and the predator ($y(t)$) in the interval $[0, 300]$.

(c) Solve the Lotka-Volterra equation by means of the Crank-Nicolson method. Describe the problems encountered and the solutions adopted.

Solution. We consider the predator-prey system defined by the Lotka-Volterra equations:

$$\begin{cases} x'(t) = x(t)(\alpha - \beta y(t)) \\ y'(t) = y(t)(\gamma x(t) - \delta) \end{cases} \tag{10}$$

with parameters $\alpha = 0.2$, $\beta = 0.01$, $\gamma = 0.004$, $\delta = 0.07$. The initial conditions are $x_0 = 19$, $y_0 = 22$ over the interval $t \in [0, 300]$. We denote the state vector as $\mathbf{u}(t) = [x(t), y(t)]^T$ and the vector field as $\mathbf{f}(t, \mathbf{u})$.

(a) **Solution via Runge-Kutta 4**
To solve the system numerically, we employed the explicit RK4. The step size was set to $h = 10^{-3}$. The generic update step for the system $\mathbf{u}' = \mathbf{f}(t, \mathbf{u})$ is given by:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{11}$$

where the four stages are computed as:

$$\begin{aligned} k_1 &= \mathbf{f}(t_n, \mathbf{u}_n) \\ k_2 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{u}_n + \frac{h}{2}k_1\right) \\ k_3 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{u}_n + \frac{h}{2}k_2\right) \\ k_4 &= \mathbf{f}\left(t_n + h, \mathbf{u}_n + hk_3\right) \end{aligned}$$

Since the method is explicit, the simulation proceeds directly without solving algebraic equations at each step.

(b) **Numerical Results**

The simulation results obtained with RK4 reveal the typical oscillatory dynamics of the Lotka-Volterra system. As shown in Figure 3, the predator population (red) lags behind the prey population (blue), creating a periodic cycle. This periodicity is further confirmed by the phase plane trajectory in Figure 4, which forms a closed loop, indicating that the system returns to its initial state without energy dissipation or gain (characteristic of conservative systems).

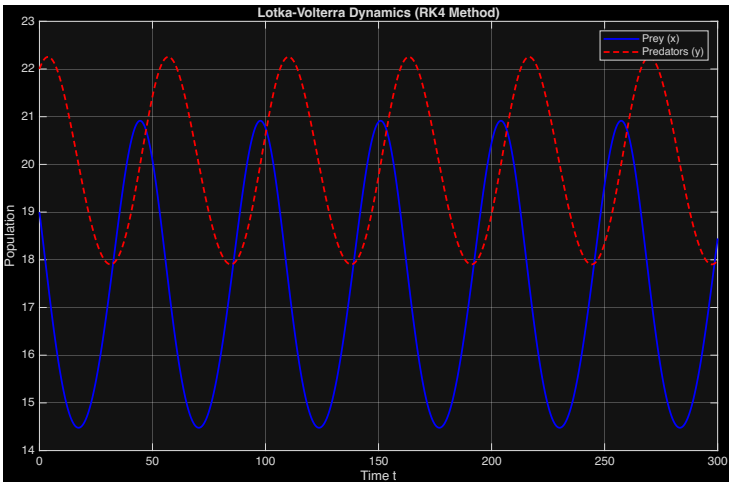


Figure 3: Time evolution of Prey $x(t)$ (blue) and Predator $y(t)$ (red) computed with RK4.

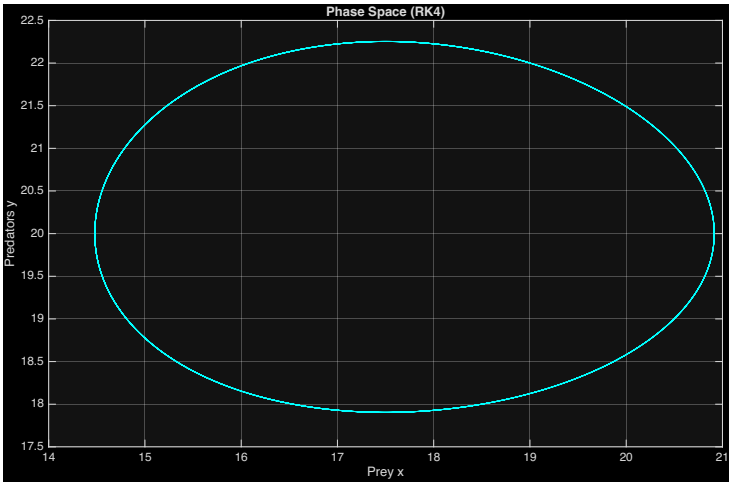


Figure 4: Phase plane trajectory $(x(t), y(t))$. The closed loop indicates periodic dynamics.

(c) **Solution via Crank-Nicolson**

- **Method formulation and Non-Linearity Issues**

  We applied the Crank-Nicolson method, which is an implicit A-Stable scheme of order 2:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \frac{h}{2}\left[\mathbf{f}(\mathbf{u}_n) + \mathbf{f}(\mathbf{u}_{n+1})\right] \tag{12}$$

  Substituting the specific Lotka-Volterra functions, the equation for the first component x_{n+1} becomes:

$$x_{n+1} = x_n + \frac{h}{2}\left[x_n(\alpha - \beta y_n) + x_{n+1}(\alpha - \beta y_{n+1})\right] \tag{13}$$

The term $x_{n+1}y_{n+1}$ makes the algebraic system for the unknown $\mathbf{u}_{n+1}$ **non-linear**. Consequently, it is impossible to isolate $\mathbf{u}_{n+1}$ using simple linear algebra operations (matrix inversion).

- **Newton Solver implementation**
  Since equation 13 is non-linear we solved it using the Newton-Raphson method. The root-finding problem at each step is defined by the residual function:

$$\mathbf{F}(\mathbf{u}_{n+1}) = \mathbf{u}_{n+1} - \mathbf{u}_n - \frac{h}{2}\left[\mathbf{f}(\mathbf{u}_n) + \mathbf{f}(\mathbf{u}_{n+1})\right] = 0 \tag{14}$$

  The solver requires the Jacobian of the residual:

$$\mathbf{J}_F(\mathbf{u}) = \mathbf{I} - \frac{h}{2}\mathbf{J}_f(\mathbf{u}) \tag{15}$$

  We explicitly derived the Jacobian of the Lotka-Volterra vector field as:

$$\mathbf{J}_f(x, y) = \begin{bmatrix} \alpha - \beta y & -\beta x \\ \gamma y & \gamma x - \delta \end{bmatrix} \tag{16}$$

  In the MATLAB implementation, the Newton loop typically converges within 3-4 iterations with a tolerance of 10^{-8}. This approach preserves the stability properties of the Crank-Nicolson method without excessive computational cost.

- **Validation of Results**
  To verify the correctness of the implicit implementation, we compared the solution obtained via Crank-Nicolson with the reference solution computed via RK4.
  Figure 5 presents a side-by-side comparison:
  - The **left panel** (Full Interval T) demonstrates the long-term stability of the implicit method, showing no amplitude decay or phase drift over 300 seconds.
  - The **right panel** (Zoom [0,50]) confirms the local accuracy: the trajectories overlap perfectly, validating the Newton-Raphson convergence at each step.

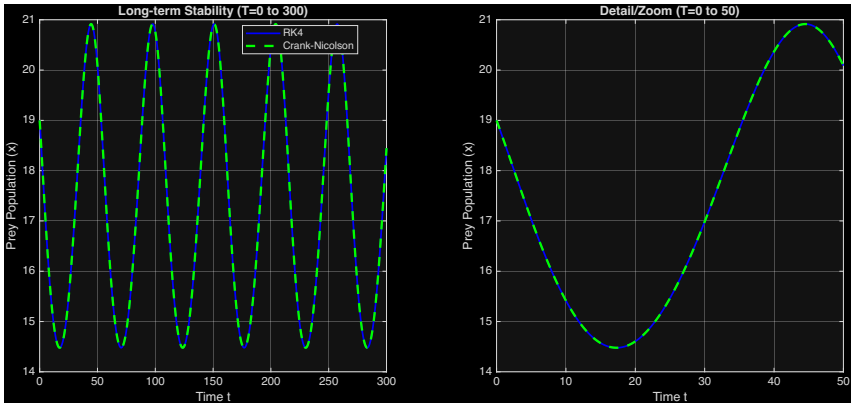


Figure 5: Comparison of the Prey population $x(t)$: Explicit RK4 (blue) vs Implicit Crank-Nicolson (green).

The script I used for the simulations and data generation is provided in the attached file `matlab_script/lotka_volterra_solver.m`.

□

Exercise 5. Assume we use the Crank-Nicolson method to solve (1) up to the final time $T = 100$. Start with $h = 10^{-3}$ and develop an adaptive timestep procedure, with tolerance 10^{-8} to arrive at the final time. Plot the error with reference to the exact solution, in both cases (constant and variable timestep). Provide a plot of the stepsize for different times. Which is the more efficient strategy? Please comment.

Solution. To address the strategy as requested we implemented an adaptive procedure based on **Richardson Extrapolation** applied to the Crank-Nicolson method (Order $p = 2$). The procedure implemented is as follows:

1. At each step t_n, we compute two candidate solutions:
   - y_1: Result of one step of size h for a rough estimate.
   - y_2: Result of two steps of size $h/2$ for a more accurate estimate.
2. We estimate the local truncation error E using the formula for order $p = 2$:
$$E \approx \frac{y_2 - y_1}{2^p - 1} = \frac{y_2 - y_1}{3} \tag{17}$$
3. **Step Selection:**
   - If $||E||_\infty \leq TOL$: The step is accepted. The solution is updated using y_2 (plus the error correction). The next step size is increased.
   - If $||E||_\infty > TOL$: The step is rejected. The step size is decreased.
4. The new step size is computed as:
$$h_{new} = h_{old} \cdot 0.9 \cdot \left(\frac{TOL}{||E||}\right)^{\frac{1}{p+1}} \tag{18}$$

Results and Discussion

- **Error Analysis**
  Figure 6 compares the global error of the two strategies with respect to the analytical solution $y(t) = (10t+1)^{-1}$. The constant step method (red line, $h = 10^{-3}$) exhibits a large initial error ($\approx 10^{-5}$), significantly exceeding the requested tolerance of 10^{-8}. This is due to the high magnitude of the derivative at $t = 0$ ($|y'(0)| = 10$), which requires a much finer resolution than 10^{-3} to be captured accurately. The error decreases later only because the solution itself decays towards zero.

  In contrast, the adaptive Crank-Nicolson strategy (blue line) successfully maintains the global error close to or below the tolerance threshold throughout the entire simulation. By automatically refining the step size, it ensures accuracy when the dynamics are fast, without requiring a globally small timestep.

- **Step Size Evolution**
  Figure 7 illustrates the evolution of $h(t)$:
  - **Initial Phase ($t < 1$):** The solver detects the rapid change in the solution and restricts h to values smaller than 10^{-3} to satisfy the local error tolerance.
  - **Asymptotic Phase ($t \gg 1$):** As $y(t) \to 0$, the derivative $y'(t) \to 0$ and the solution becomes nearly flat. The adaptive algorithm correctly identifies that the problem has become numerically "easy" and progressively increases h, eventually reaching step sizes larger than 1 (orders of magnitude larger than the initial steps).

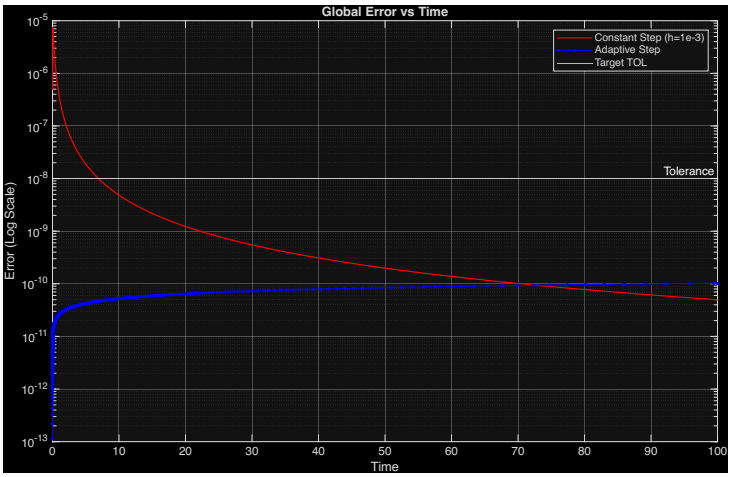


Figure 6: Global error comparison: Constant Step (Red) vs Adaptive Step (Blue).

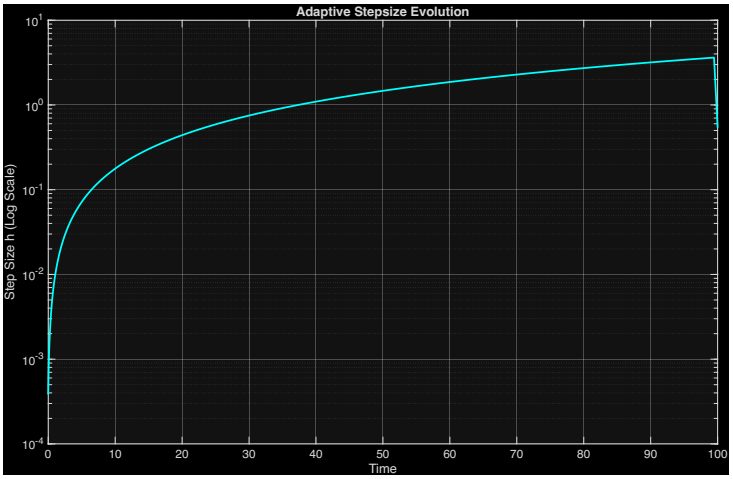


Figure 7: Evolution of the step size h over time.

- **Efficiency Comparison**
  The adaptive strategy is significantly more efficient. Table 4 summarizes the performance of both the constant step and adaptive step methods. To achieve the initial accuracy of the adaptive method using a fixed step, one would require an extremely small h for the entire duration $T = 100$. The adaptive solver minimizes computational effort by allocating resources only where necessary and accelerating significantly towards the end.

Table 4: Efficiency Comparison

Method	**Steps**	**Max Error**	**CPU Time**
Constant	100001	7.407750e-06	0.0371 s
Adaptive	705	1.027390e-10	0.0105 s

As the results show, the adaptive method achieved a lower error with significantly fewer steps and a fraction of the CPU time. In conclusion, the adaptive procedure maintains the target accuracy with fewer steps compared to the fixed step approach, optimizing the computational cost.

The script I used for the simulations and data generation is provided in the attached file `matlab_script/adaptive_cn.m`. □